Session 24 Teamwork — A Real Light/Dark Theme via React Context

Context

Work in your team (six teams of four students). LIVE already restyled every page with Tailwind and `darkMode: "class"` — but nothing in the real app ever adds that `dark` class to `<html>` yet, so every `dark:` utility you've typed so far has never actually activated. Your job: build the real mechanism that controls it, shared across the entire app, through React Context.

Goal

A real `ThemeProvider` that starts from the user's own real OS preference, a real `useTheme()` hook, a real visible toggle button wired into the nav bar every page already shares, and Issue Detail restyled with the same real `Card` / `StatusBadge` primitives every other page now uses.

What to build

- A real `ThemeContext.tsx` exporting a `ThemeProvider` component. Its own state starts from the real system preference (`window.matchMedia("(prefers-color-scheme: dark)")`), read exactly once, then the Context owns every change after that — think carefully about why this needs TWO separate `useEffect` calls, not one.
- A real `useTheme()` hook that throws a specific, loud error if it is ever called outside the provider, instead of silently returning something wrong.
- A real `ThemeToggle` button, calling `toggleTheme()`, labeled with the DESTINATION theme (what clicking it switches TO), not the current one.
- `ThemeProvider` / `ThemeToggle` wired into the root layout (`web/src/app/layout.tsx`) — think about which parts of that file can stay a Server Component and which parts genuinely need `"use client"`, and where exactly that boundary should start.
- Issue Detail (`web/src/app/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx`) restyled with `Card` / `StatusBadge`, matching the same visual language Project Detail already uses.

Rules

Allowed: everything already shown LIVE this session (Tailwind utilities, `darkMode: "class"`, `Button` / `Card` / `Badge`), plus React Context (`createContext` / `useContext` / `useEffect` / `useState`) from PowerUp, plus the real official React and Next.js documentation for Context and Client Components.

Not yet: no persisting the theme choice across a reload — that's this session's own homework CHALLENGE, deliberately left open here. No Server Actions, no Redux Toolkit, no real authentication.

Test it

inside api/

BASH

```
PORT=3000 npm run start
```

inside web/ (second terminal)

BASH

```
npm run dev
```

in a third terminal

BASH

```
npx tsc --noEmit
```

Then, in a real browser: open `http://localhost:3001`, click the real toggle button, confirm the whole page switches instantly and the button's own label flips; open an issue's detail page and confirm it now matches every other page's visual language.

Done when

- Clicking the real toggle button switches the ENTIRE app between light and dark instantly, not just the nav bar.
- `useTheme()` used anywhere outside `ThemeProvider` throws your own specific error, not a silent crash somewhere else.
- Issue Detail renders inside a single `Card`, with its status shown via `StatusBadge`.
- `npx tsc --noEmit` exits clean.

Working time

About 51 minutes total (TW-01..05).